Recording Train Service Status using Auckland Transport's API

Clifford Dela Cruz

3/18/26

Document Version Control

Version	Date	Revised By	Summary of Changes
1.0	18/03/2026	Clifford	
1.1	08/04/2026	Clifford	Added version control, added table of contents, updated flowchart for 'Partially Running'

Contents

Objective

The objective of this project is to capture and analyse instances where Auckland's train services (Eastern, Southern, Onehunga, and Western lines) stop operating due to unplanned faults or scheduled maintenance. As a regular commuter, I aim to build a dataset that helps explain the frequency and timing of these disruptions, particularly those occurring during peak office hours. The intention is not to criticise Auckland Transport, but to understand the underlying patterns and identify how often these service interruptions occur.

Challenges encountered

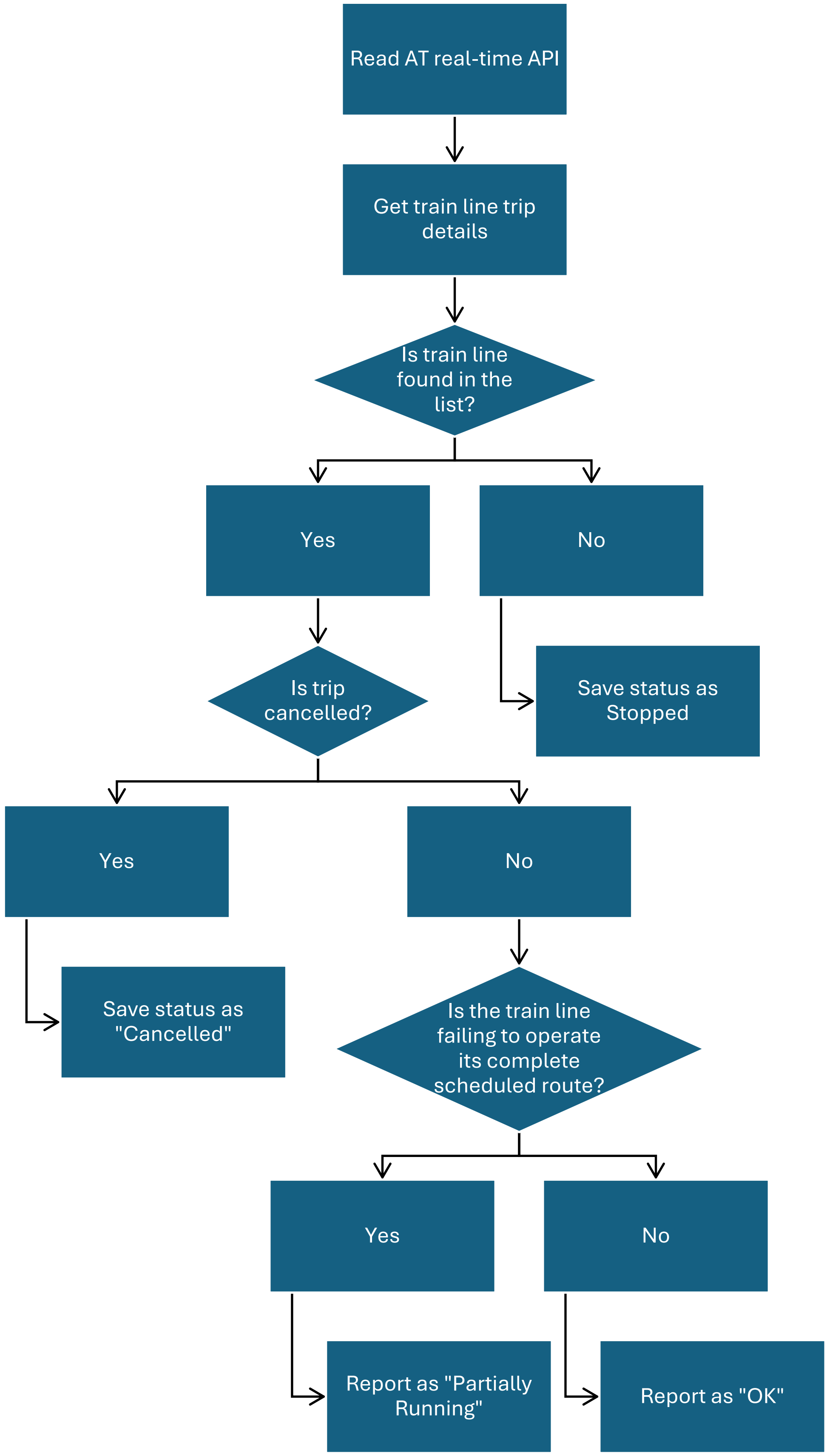
Initially, I explored the possibility of extracting disruption information through web scraping from the Auckland Transport train line status page. However, this approach was not feasible. The website relies on internal API endpoints that are not publicly accessible, meaning the data displayed on the site cannot be retrieved programmatically.

The Auckland Transport public API also presents limitations. It does not provide a direct feed of line-level disruptions or maintenance shutdowns. Instead, the real-time API only reports information at the *trip* level—specifically, whether a trip is running or cancelled. To work around this constraint, I ingest the full real-time feed and determine whether a train line has active trips. If a line has no active trips, it is classified as "stopped."

To detect partial closures (for example, when only a segment such as Puhinui–Pukekohe is closed for maintenance), each trip is evaluated based on its start and end points. By checking the trip headsigns, I can determine whether the full route is operating or whether only part of the line is active.

Flowchart

This is a high-level description of the program flow



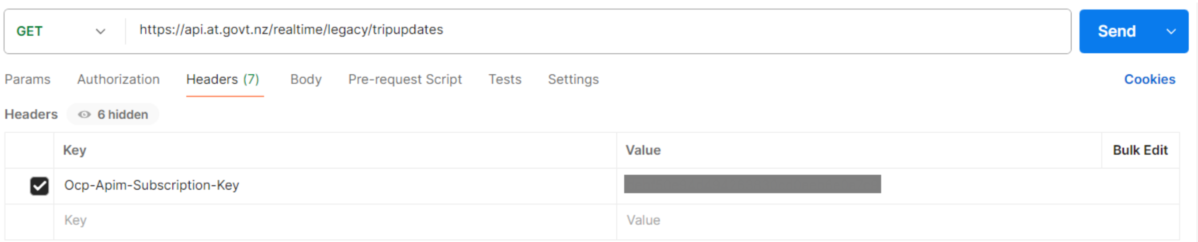
APIs used

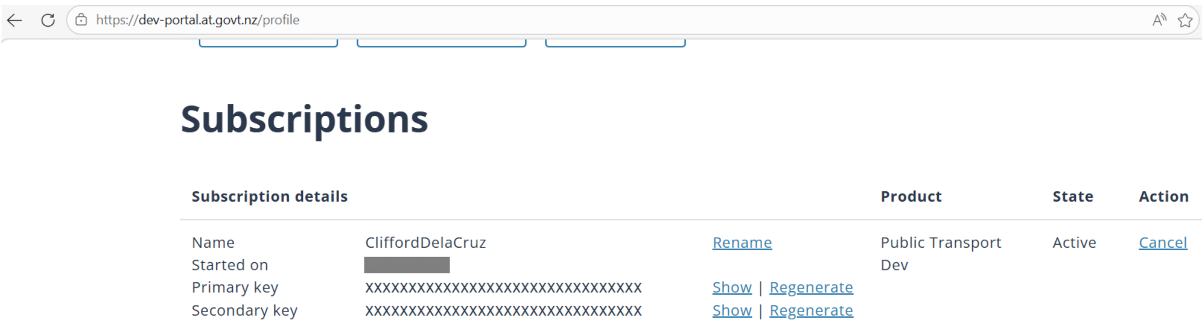
The APIs I used are the following:

1. Real Time compat – Trip Updates: this is used to take in all the train service updates. This will provide the information such as Trip ID, the Route ID, the status of the trip.

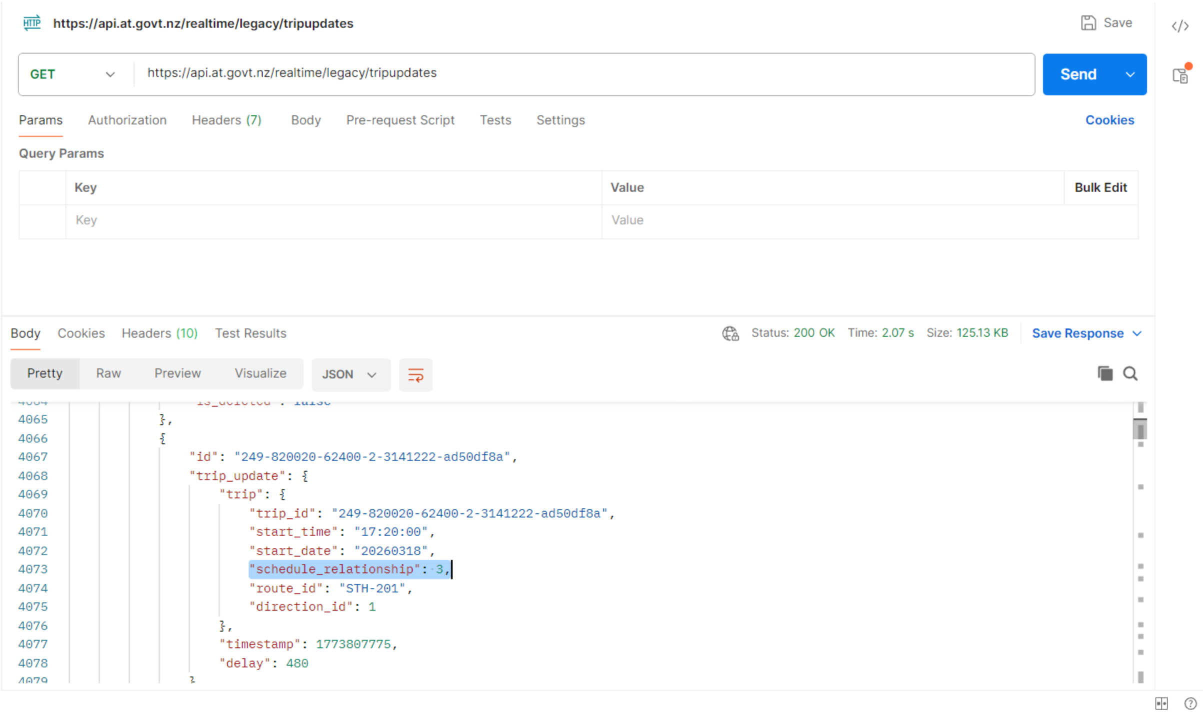
   For this project, we are focusing on route ID and trip ID, as those contains the train line information as well as the trip information respectively.
   URL: https://api.at.govt.nz/realtime/legacy/tripupdates[?callback][&tripid][&vehicleid]
   Resource: https://dev-portal.at.govt.nz/api-details#api=gtfs-realtime-compat&operation=get_trip_updates

   Prerequisites – need an API key (Ocp-Apim-Subscription-Key) which can be acquired by registering in https://dev-portal.at.govt.nz/





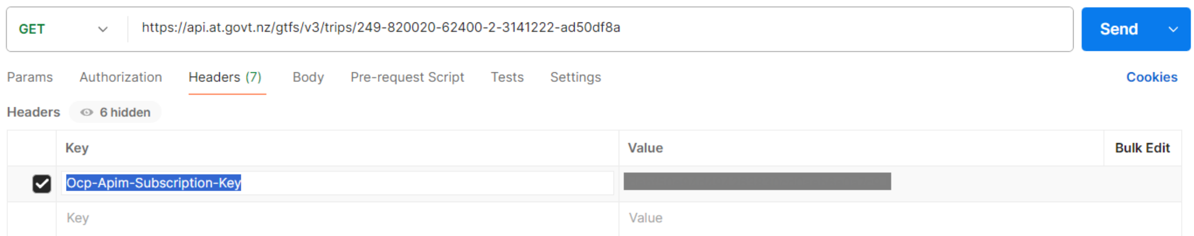
Sample images of key header input in Postman and subscription information

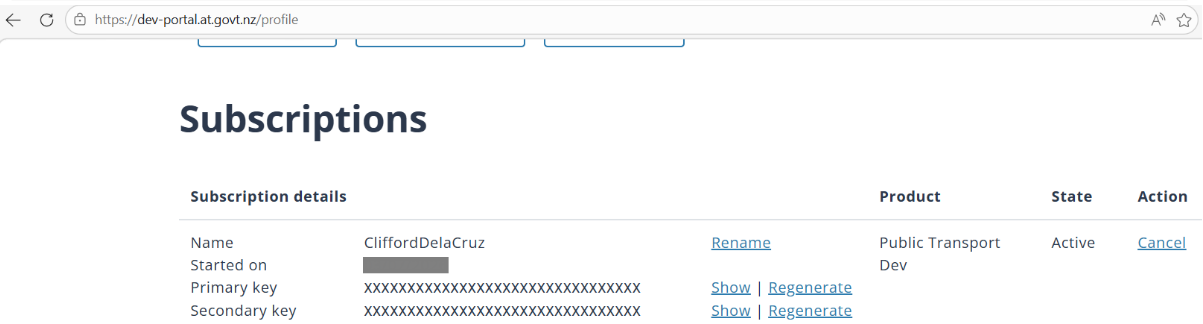


Example 1.1 This image shows one result of a trip in Southern line that is cancelled.
(schedule relationship = 3 means trip is cancelled)

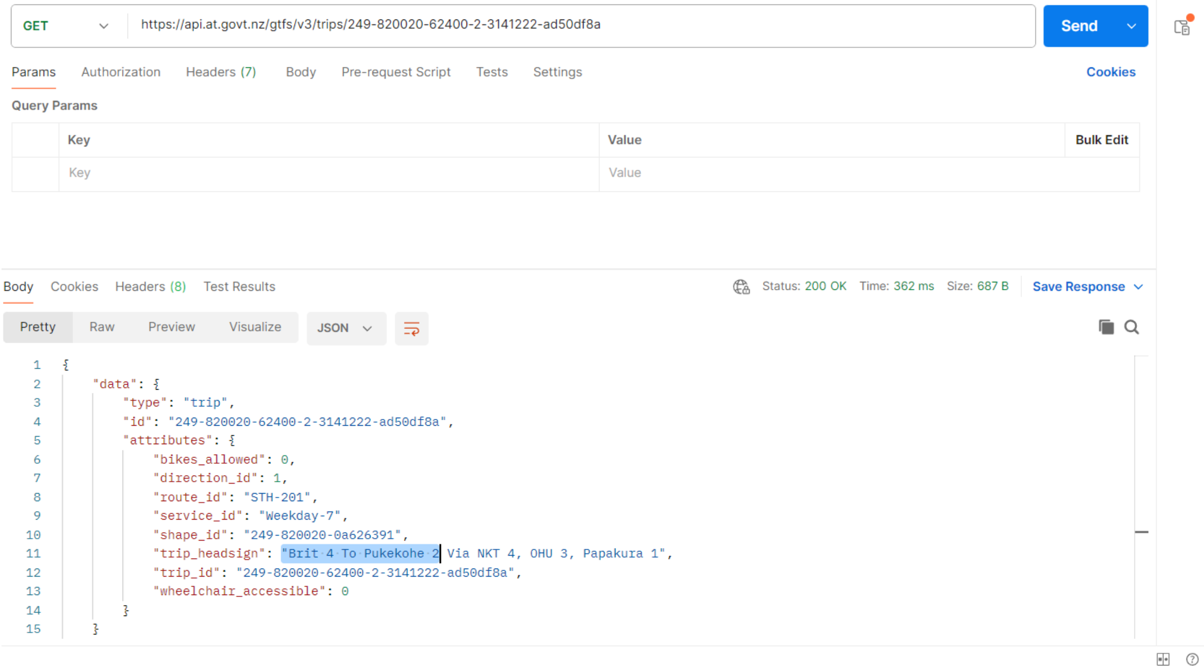
2. General Transit Feed V3 API – Trip by ID: this is used to identify the trip ID to indicate the services covered in that trip. Example, trip id 246-850001-30660-2-4234101-0620e5fd is an Eastern line service trip, which is a trip from Manukau to Britomart
URL: https://api.at.govt.nz/gtfs/v3/trips/{id}
Resource: https://dev-portal.at.govt.nz/api-details#api=gtfs-api&operation=get-trips-id

Prerequisites – need an API key (Ocp-Apim-Subscription-Key) which can be acquired by registering in https://dev-portal.at.govt.nz/





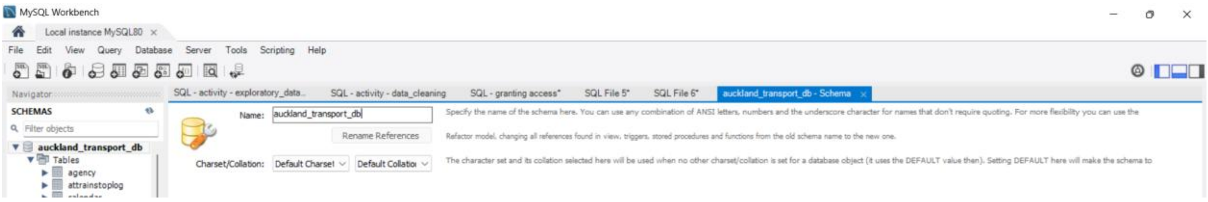
Sample images of key header input in Postman and subscription information



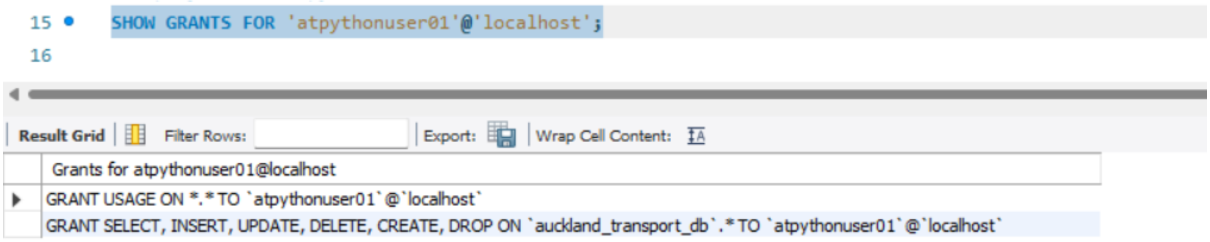
Example 2.1 This image shows that the trip information that it is a train service from Britomart to Pukekohe

Database setup

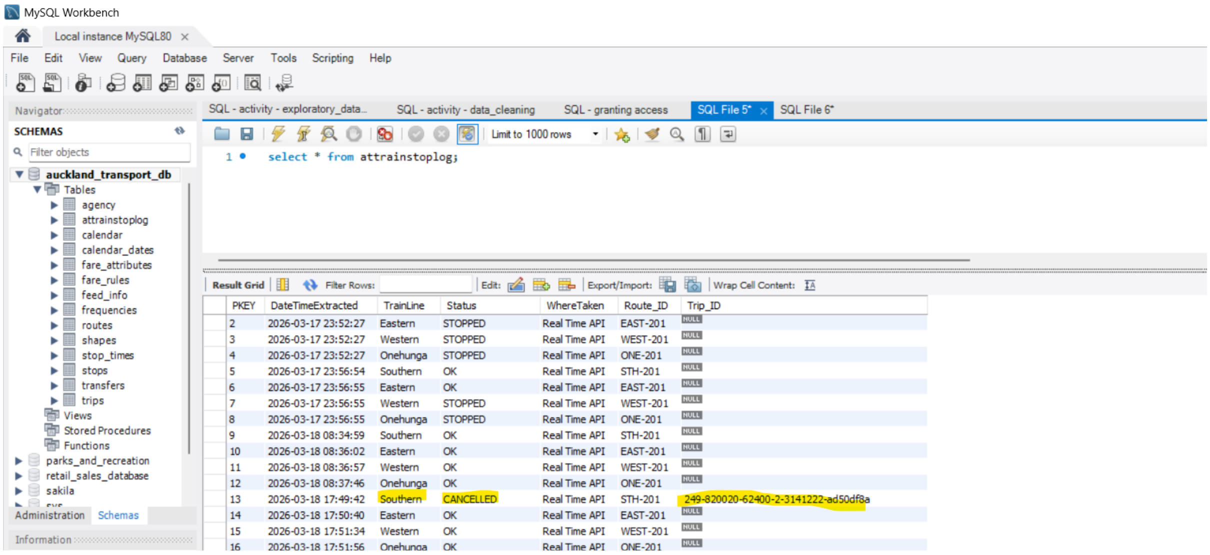
I used MySQL and created a schema "auckland_transport_db"



I created an account that can access the database and granted CREATE, DROP, SELECT, INSERT, UPDATE, DELETE role. This role will then be used in the program to be able to create and update a table "attrainstoplog".

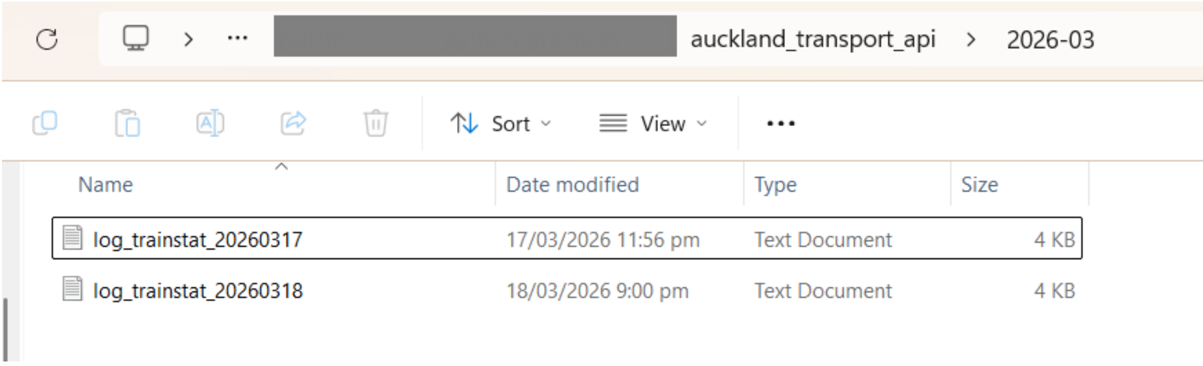


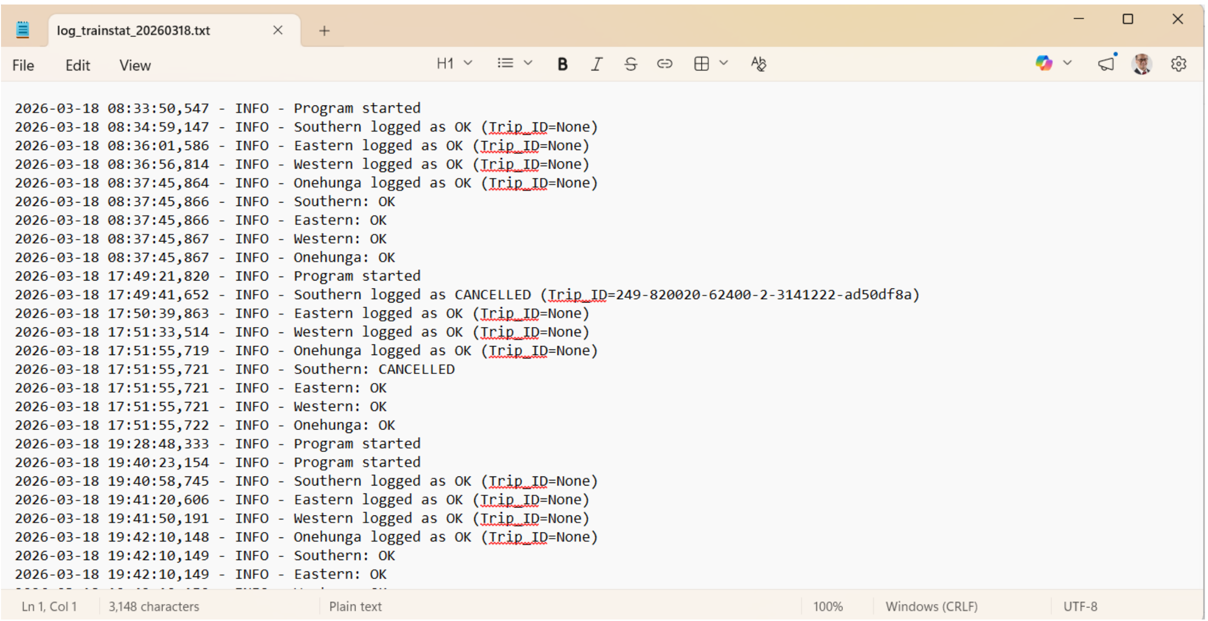
Running the "attrainstoplog" will show the information on the strain line during the program call



PKEY	DateTimeExtracted	TrainLine	Status	WhereTaken	Route_ID	Trip_ID
2	2026-03-17 23:52:27	Eastern	STOPPED	Real Time API	EAST-201	NULL
3	2026-03-17 23:52:27	Western	STOPPED	Real Time API	WEST-201	NULL
4	2026-03-17 23:52:27	Onehunga	STOPPED	Real Time API	ONE-201	NULL
5	2026-03-17 23:56:54	Southern	OK	Real Time API	STH-201	NULL
6	2026-03-17 23:56:55	Eastern	OK	Real Time API	EAST-201	NULL
7	2026-03-17 23:56:55	Western	STOPPED	Real Time API	WEST-201	NULL
8	2026-03-17 23:56:55	Onehunga	STOPPED	Real Time API	ONE-201	NULL
9	2026-03-18 08:34:59	Southern	OK	Real Time API	STH-201	NULL
10	2026-03-18 08:36:02	Eastern	OK	Real Time API	EAST-201	NULL
11	2026-03-18 08:36:57	Western	OK	Real Time API	WEST-201	NULL
12	2026-03-18 08:37:46	Onehunga	OK	Real Time API	ONE-201	NULL
13	2026-03-18 17:49:42	Southern	CANCELLED	Real Time API	STH-201	249-820020-62400-2-3141222-ad50df8a
14	2026-03-18 17:50:40	Eastern	OK	Real Time API	EAST-201	NULL
15	2026-03-18 17:51:34	Western	OK	Real Time API	WEST-201	NULL
16	2026-03-18 17:51:56	Onehunga	OK	Real Time API	ONE-201	NULL

Logs are also created/updated to record the start of program run as well as the train status






```
2026-03-18 08:33:50,547 - INFO - Program started
2026-03-18 08:34:59,147 - INFO - Southern logged as OK (Trip_ID=None)
2026-03-18 08:36:01,586 - INFO - Eastern logged as OK (Trip_ID=None)
2026-03-18 08:36:56,814 - INFO - Western logged as OK (Trip_ID=None)
2026-03-18 08:37:45,864 - INFO - Onehunga logged as OK (Trip_ID=None)
2026-03-18 08:37:45,866 - INFO - Southern: OK
2026-03-18 08:37:45,866 - INFO - Eastern: OK
2026-03-18 08:37:45,867 - INFO - Western: OK
2026-03-18 08:37:45,867 - INFO - Onehunga: OK
2026-03-18 17:49:21,820 - INFO - Program started
2026-03-18 17:49:41,652 - INFO - Southern logged as CANCELLED (Trip_ID=249-820020-62400-2-3141222-ad50df8a)
2026-03-18 17:50:39,863 - INFO - Eastern logged as OK (Trip_ID=None)
2026-03-18 17:51:33,514 - INFO - Western logged as OK (Trip_ID=None)
2026-03-18 17:51:55,719 - INFO - Onehunga logged as OK (Trip_ID=None)
2026-03-18 17:51:55,721 - INFO - Southern: CANCELLED
2026-03-18 17:51:55,721 - INFO - Eastern: OK
2026-03-18 17:51:55,721 - INFO - Western: OK
2026-03-18 17:51:55,722 - INFO - Onehunga: OK
2026-03-18 19:28:48,333 - INFO - Program started
2026-03-18 19:40:23,154 - INFO - Program started
2026-03-18 19:40:58,745 - INFO - Southern logged as OK (Trip_ID=None)
2026-03-18 19:41:20,606 - INFO - Eastern logged as OK (Trip_ID=None)
2026-03-18 19:41:50,191 - INFO - Western logged as OK (Trip_ID=None)
2026-03-18 19:42:10,148 - INFO - Onehunga logged as OK (Trip_ID=None)
2026-03-18 19:42:10,149 - INFO - Southern: OK
2026-03-18 19:42:10,149 - INFO - Eastern: OK
```

Further improvements and recommendations

To maximise the reliability and continuity of data collection, the program is best deployed in a server-based environment rather than run manually on a local machine. In the future, the solution will be migrated to a cloud platform (Azure), where the Python process can be executed on a scheduled basis and the resulting data automatically stored in a managed database service.

Additional links for reference
https://dev-portal.at.govt.nz/realtime-api
https://gtfs.org/documentation/schedule/reference/